In Java, the **super** keyword is used in three main contexts — all related to working with inheritance:

1. Accessing the Parent Class Constructor

If a subclass wants to call its **parent class constructor**, it can use:

```
super(arguments);
```

- **Must be** the **first statement** inside the subclass constructor.
- Useful when you want to initialize parent class fields before doing subclass-specific work.

Example:

```
class Parent {
    Parent(String name) {
        System.out.println("Parent constructor: " + name);
    }
}

class Child extends Parent {
    Child() {
///String s;
        super("Tasniah"); // calls Parent constructor
        System.out.println("Child constructor");
    }
}

public class Main {
    public static void main(String[] args) {
 Child c= new Child();
    }
}
```

Output:

Parent constructor: Tasniah
Child constructor

2. Accessing Parent Class Methods

When a method in the child class overrides a method in the parent, you can still call the **parent's original method** using super.

super.methodName();

Example:

```
class Parent {
    void display() {
        System.out.println("Parent display");
    }
}

class Child extends Parent {
    void display() {
       super.display(); // calls parent display
        System.out.println("Child display");
    }
}
```

ob.display();

3. Accessing Parent Class Variables

If a subclass has a **field with the same name** as the parent class (variable shadowing), super.variableName refers to the parent's variable.

Example:

```
class Parent {
    int num = 100;
}

class Child extends Parent {
    int num = 200;

    void show() {
        System.out.println(super.num); // parent variable
        System.out.println(num);       // child variable
    }
}
ob.num;
```

Summary Table

Usage	Syntax	Purpose

Call parent constructor	super(args)	Initialize parent class before child
Call parent method	super.method()	Access overridden parent method
Access parent variable	super.variable	Resolve shadowed variables

https://ideone.com/AhvuJf

Problem 1. You are tasked with creating a small Java program to demonstrate the use of the super keyword in inheritance. Design a parent class Person that contains a variable name, a constructor to initialize it, and a method introduce() that prints the person's name. Then, create a child class Employee that extends Person but also has its own name variable to store a nickname, thereby shadowing the parent's variable. The Employee constructor should take both the full name (for the parent) and the nickname (for the child) and use super to call the parent's constructor with the full name. Override the introduce() method in Employee so that it first calls the parent's version of introduce() using super.introduce(), then prints the nickname. Additionally, create a showNames() method in Employee that uses super.name to access the parent's name variable and this.name for the child's nickname. In the main() method, create an Employee object and demonstrate all three uses of the super keyword: calling the parent constructor, invoking a parent method from an overridden method, and accessing a shadowed variable from the parent class.

Dynamic Method Dispatch

Dynamic Method Dispatch in Java is the process by which a method call to an overridden method is resolved at runtime rather than at compile time.

- It's also known as **runtime polymorphism**.
- It allows Java to decide **which version of the overridden method to call based on the object type that is being referred to**, not the reference type.

Main Points:

1. Achieved using method overriding (same method in superclass and subclass).

2. Reference variable of parent class can hold the object of child class.

3. At runtime, the child class method is executed if the object is of child type.

   https://ideone.com/4p7rbS

- The **reference variable** a is of type Animal.

- At **runtime**, JVM decides whether to call Dog's, Cat's, or Animal's sound() method depending on the object assigned.

That’s **Dynamic Method Dispatch** (runtime polymorphism).